SolTrace Result API Redesign

Reducing data movement and enabling flexible result types

July 12, 2026

Motivation

Problem: One rigid result type

Before: `report_simulation` always builds a full ray history.

- Every hit → heap-allocated `shared_ptr<InteractionRecord>`
- All ray paths grouped into `RayRecord` objects
- Per-element `element_view` map always constructed
- GPU runner downloads the full compacted hit buffer unconditionally

Many use cases only need a subset of this data, e.g.:

- Flux per element (no per-ray records needed)
- Hits on a specific subset of elements
- Raw hit buffer for custom post-processing

Old API

```
// SimulationRunner
virtual RunnerStatus setup_simulation(
    const SimulationData *data) = 0;

virtual RunnerStatus report_simulation(
    SimulationResult *result,   // single concrete type
    int level_spec) = 0;        // unused parameter

// Caller
SimulationResult result;
runner.setup_simulation(&data);
runner.run_simulation();
runner.report_simulation(&result, 0);
```

Issues

- No way to request a lighter-weight result
- Runner cannot configure GPU buffers for the desired output
- `level_spec` was unused but required

Result Type Hierarchy

Step 1: Abstract base + ResultType enum

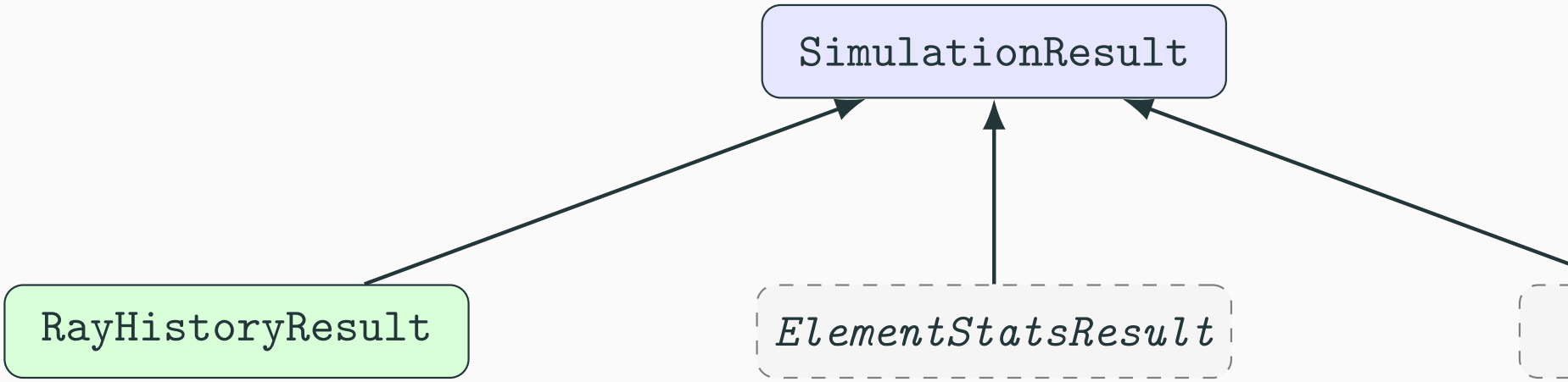
```
enum class ResultType { RAY_HISTORY, ELEMENT_STATS, RAW_HITS };

class SimulationResult {            // abstract base (was
    AbstractSimResult)
public:
    virtual ~SimulationResult() = default;
    virtual ResultType get_result_type() const = 0;

    // Sun metadata common to all result types
    void set_sun_ray_count(uint_fast64_t n);
    void set_sun_dimensions(double w, double h);
    void set_sun_A_box(double A);
    // ...
protected:
    SimulationResult() = default;
};

class RayHistoryResult : public SimulationResult {  // was
    SimulationResult
public:
    ResultType get_result_type() const override
        { return ResultType::RAY_HISTORY; }
```

Result type hierarchy



- `SimulationResult` carries sun metadata — no cast needed to set it
- `RayHistoryResult` holds full per-ray interaction history
- Dashed types are future result shapes, not yet implemented

Result Spec Hierarchy

Step 2: `ResultSpec` — configuration + factory

```
struct ResultSpec {
    virtual ResultType get_type() const = 0;

    // Factory: creates the matching empty result container
    virtual std::unique_ptr<SimulationResult> create_result() const = 0;

    // Configuration queries  default = no filter (record everything)
    virtual std::optional<std::set<element_id>>
        get_element_filter() const { return std::nullopt; }
    virtual std::optional<std::set<RayEvent>>
        get_event_filter()   const { return std::nullopt; }
};

struct RayHistorySpec : ResultSpec {
    std::optional<std::set<element_id>> element_filter;
    std::optional<std::set<RayEvent>>   event_filter;

    ResultType get_type() const override { return ResultType::
     RAY_HISTORY; }
    std::unique_ptr<SimulationResult> create_result() const override
        { return std::make_unique<RayHistoryResult>(); }
    // overrides for get_element_filter(), get_event_filter() ...
```

Why separate spec from result?

ResultSpec (input / configuration)

- Passed to `setup_simulation()`
- Runner reads it to configure GPU buffers, filters before any ray is traced
- Owns the factory method
- Short-lived; caller keeps it

SimulationResult (output / data)

- Created by `create_result()`
- Passed to `report_simulation()`
- Runner populates it after tracing
- Long-lived; caller owns and queries it

Key property
Type mismatch between setup and report is eliminated: the spec that configures the runner is the same object that creates the result container.

Updated Runner Interface

Updated `SimulationRunner` interface

```
class SimulationRunner {
public:
    // setup_simulation now receives the full spec (default =
     RayHistorySpec)
    virtual RunnerStatus setup_simulation(
        const SimulationData *data,
        const ResultSpec     &spec = RayHistorySpec{}) = 0;

    virtual RunnerStatus run_simulation() = 0;

    // Factory: create the result type this runner was configured for
    virtual std::unique_ptr<SimulationResult> create_result() = 0;

    // report_simulation accepts the abstract base  no level_spec
    virtual RunnerStatus report_simulation(
        SimulationResult *result,
        int               level_spec = 0) = 0;  // default preserves
     compat
};
```

- Default argument on `setup_simulation` keeps all existing call

New call pattern

```
//  Filter to two specific elements
RayHistorySpec spec;
spec.element_filter = { receiver_id, target_id };

runner.setup_simulation(&data, spec);   // runner configures buffers/
    filters
runner.run_simulation();

// Two equivalent ways to obtain the result container:
auto result = spec.create_result();     // caller still holds spec
// -- or --
auto result = runner.create_result();   // runner uses stored ResultType

runner.report_simulation(result.get());

//  Existing code  unchanged
runner.setup_simulation(&data);         // default RayHistorySpec, no
    filter
runner.run_simulation();
SimulationResult *r = ...;
runner.report_simulation(r, 0);         // legacy two-arg form still
    compiles
```

Runner Implementation

Inside a runner: setup & report

```
// setup_simulation  extract what the runner needs from the spec
RunnerStatus NativeRunner::setup_simulation(
    const SimulationData *data,
    const ResultSpec     &spec)
{
    m_result_type = spec.get_type();
    // future: m_element_filter = spec.get_element_filter();
    // ... existing GPU/CPU setup ...
}

// create_result  runner-side factory (delegates to ResultType switch)
std::unique_ptr<SimulationResult> NativeRunner::create_result()
{
    switch (m_result_type) {
        case ResultType::RAY_HISTORY: return make_unique<
     RayHistoryResult>();
        default:                      return nullptr;
    }
}

// report_simulation  safe downcast, returns ERROR on type mismatch
RunnerStatus NativeRunner::report_simulation(
```

Summary

Summary of changes

	Before	After
Result base	(none)	`SimulationRes`
Concrete result	`SimulationResult`	`RayHistoryRes`
Runner config	`ResultType` enum	`ResultSpec` hie
Factory	(none)	`spec.create_r`
		`runner.create`
`setup_simulation`	`(SimulationData*)`	`(SimulationDa`
`report_simulation`	`(SimulationResult*, int)`	`(SimulationRe`
Type mismatch	Runtime only	Eliminated at b
Backward compat	—	All call sites un

Add `case ResultType::ELEMENT_STATS` to each runner's `create_result()` and `report_simulation()`

No changes required to:

- The `SimulationRunner` interface
- Existing spec or result classes
- Any existing call sites